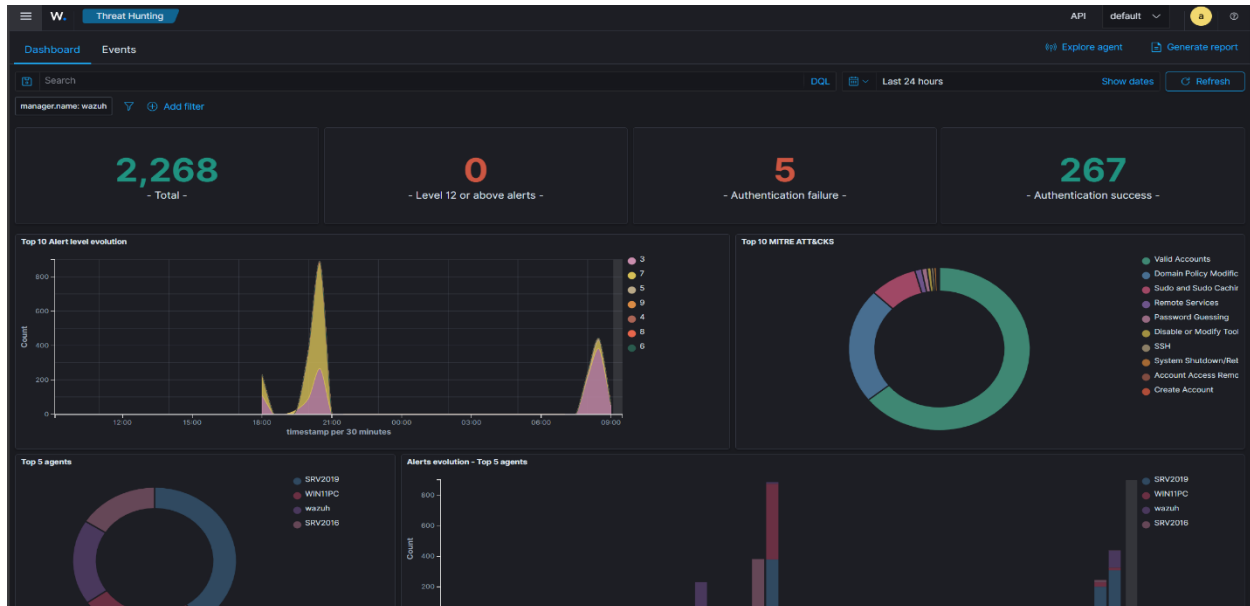
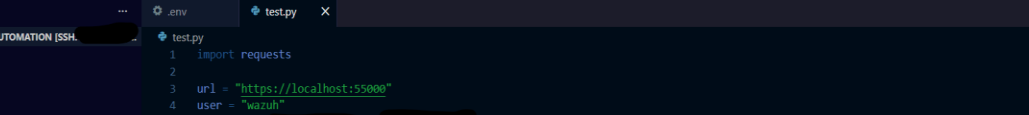


Wazuh Python Automation with MCP



This project covers the deployment of a Wazuh SIEM, the development of Python-based automation for security operations tasks, and the integration of Model Context Protocol (MCP) to enable AI-assisted interaction with security data. The solution provides automated agent inventory management, endpoint health monitoring, alert analysis, SOC reporting, and natural language access to security information through custom MCP tools.

Part 1: Python Automation



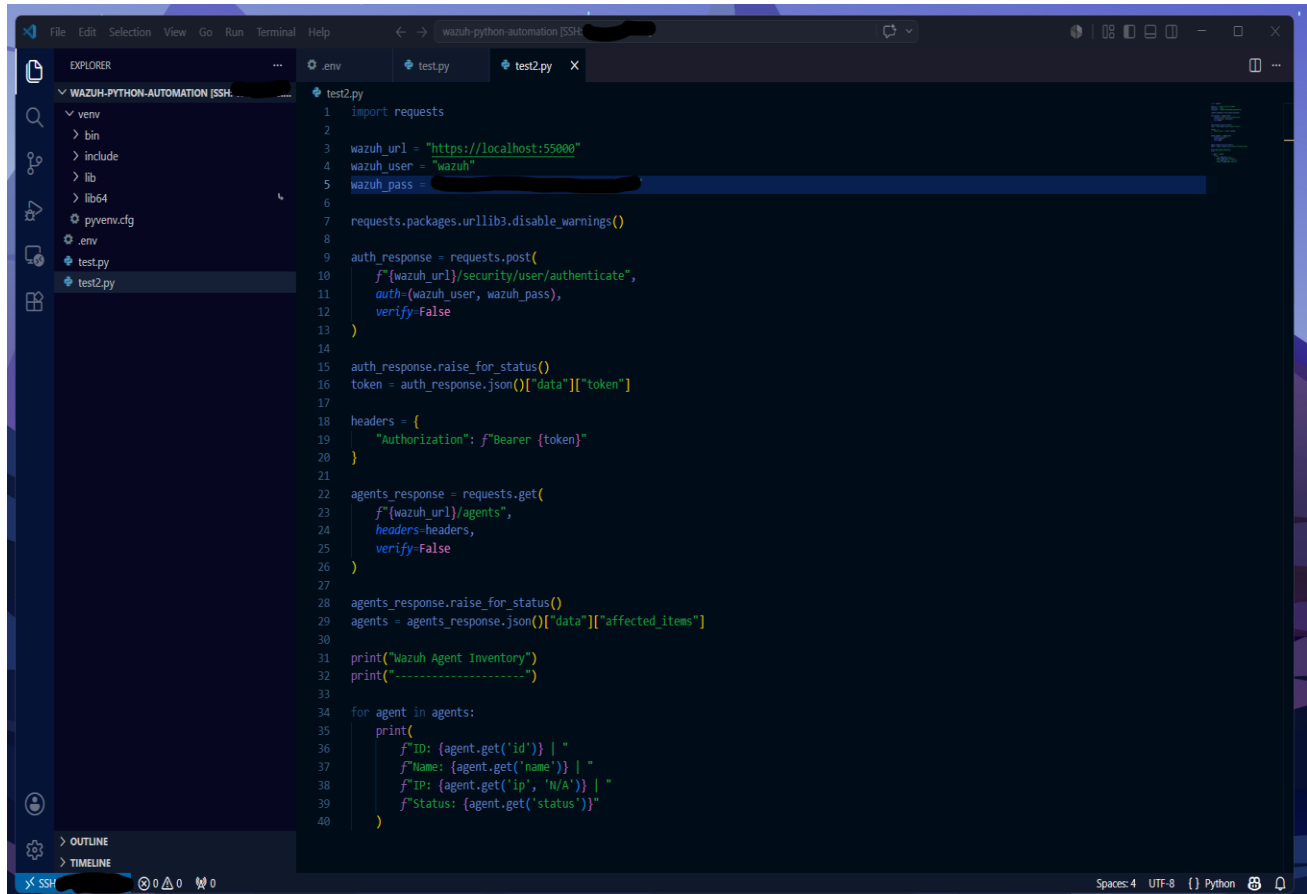
The screenshot shows a VS Code editor with a Python script named `test.py`. The script uses the `requests` library to send a POST request to a local host. The Explorer sidebar on the left shows the file structure of the project, including a `.venv` directory. The script content is as follows:

```
1 import requests
2
3 url = "https://localhost:55000"
4 user = "wazuh"
5 password = "wazuh"
6
7 requests.packages.urllib3.disable_warnings()
8
9 auth_response = requests.post(
10     f'{url}/security/user/authenticate',
11     auth=(user, password),
12     verify=False
13 )
14
15 print(auth_response.status_code)
16 print(auth_response.text)
```

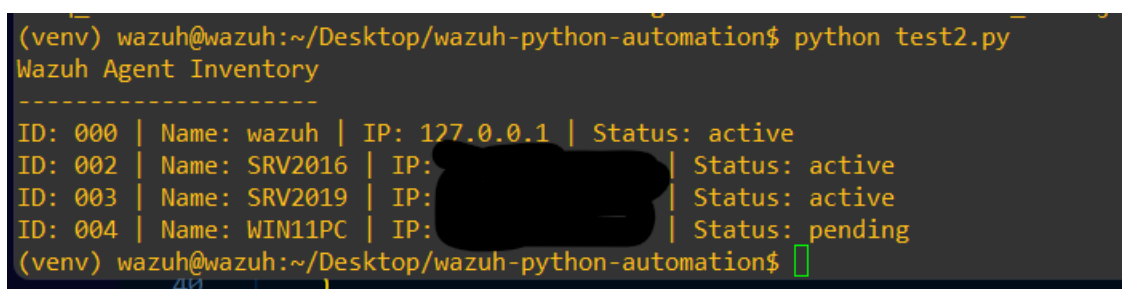
```
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ python test.py
200
{"data": {"token": "XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"},
}

(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$
```

The first task I did was configure SSH access into my ubuntu VM where the wazuh server is hosted in VS code, so I can edit my code easier. I first created a test python script for basic wazuh API authentication. Then in my SSH session in PuTTY into my VM, I typed **python test.py**, with it returning a 200 code, with the token blurred out, confirming it works.



```
1 import requests
2
3 wazuh_url = "https://localhost:55000"
4 wazuh_user = "wazuh"
5 wazuh_pass = "wazuh"
6
7 requests.packages.urllib3.disable_warnings()
8
9 auth_response = requests.post(
10     f"{wazuh_url}/security/user/authenticate",
11     auth=(wazuh_user, wazuh_pass),
12     verify=False
13 )
14
15 auth_response.raise_for_status()
16 token = auth_response.json()["data"]["token"]
17
18 headers = {
19     "Authorization": f"Bearer {token}"
20 }
21
22 agents_response = requests.get(
23     f"{wazuh_url}/agents",
24     headers=headers,
25     verify=False
26 )
27
28 agents_response.raise_for_status()
29 agents = agents_response.json()["data"]["affected_items"]
30
31 print("Wazuh Agent Inventory")
32 print("-----")
33
34 for agent in agents:
35     print(
36         f"ID: {agent.get('id')} | "
37         f"Name: {agent.get('name')} | "
38         f"IP: {agent.get('ip', 'N/A')} | "
39         f"Status: {agent.get('status')}"
40     )
```



```
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ python test2.py
Wazuh Agent Inventory
-----
ID: 000 | Name: wazuh | IP: 127.0.0.1 | Status: active
ID: 002 | Name: SRV2016 | IP: [REDACTED] | Status: active
ID: 003 | Name: SRV2019 | IP: [REDACTED] | Status: active
ID: 004 | Name: WIN11PC | IP: [REDACTED] | Status: pending
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$
```

Next, I created an automated reporting function that analyzes agent status information and identifies systems that are no longer reporting as active. This functionality assists security administrators in quickly identifying disconnected, pending, or potentially unhealthy endpoints requiring investigation. Back to the SSH session, running the script with **python test2** confirms it is working.

```
.env test.py agent-inventory.py offline-agents.py X
offline-agents.py
1 import requests
2
3 wazuh_url = "https://localhost:55000"
4 wazuh_user = "wazuh"
5 wazuh_pass = "wazuh"
6
7 requests.packages.urllib3.disable_warnings()
8
9 auth_response = requests.post(
10     f"{wazuh_url}/security/user/authenticate",
11     auth=(wazuh_user, wazuh_pass),
12     verify=False
13 )
14
15 auth_response.raise_for_status()
16 token = auth_response.json()["data"]["token"]
17
18 headers = {
19     "Authorization": f"Bearer {token}"
20 }
21
22 agents_response = requests.get(
23     f"{wazuh_url}/agents",
24     headers=headers,
25     verify=False
26 )
27
28 agents_response.raise_for_status()
29 agents = agents_response.json()["data"]["affected_items"]
30
31 print("\nOffline Agent Report")
32 print("=" * 50)
33
34 offline_found = False
35
36 for agent in agents:
37     status = agent.get("status", "").lower()
38
39     if status != "active":
40         offline_found = True
41
42         print(
43             f"ID: {agent.get('id')} | "
44             f"Name: {agent.get('name')} | "
45             f"IP: {agent.get('ip', 'N/A')} | "
46             f>Status: {agent.get('status')}"
47         )
48
49 if not offline_found:
50     print("All agents are active.")
```

```
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ python offline-agents.py
Offline Agent Report
=====
ID: 004 | Name: WIN11PC | IP: [REDACTED] | Status: pending
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$
```

Next, I developed an automated Security Operations Center (SOC) reporting script that consolidates agent health information, offline endpoint status, alert activity, and security findings into a single daily report.

```

high-severity-alerts.py
1  import json
2  from pathlib import Path
3
4  alerts_file = Path("/var/ossec/logs/alerts/alerts.json")
5  minimum_level = 10
6  limit = 20
7
8  print("\nHigh Severity Alerts")
9  print("=" * 70)
10
11 if not alerts_file.exists():
12     print(f"Alerts file not found: {alerts_file}")
13     raise SystemExit(1)
14
15 matches = []
16
17 with alerts_file.open("r", encoding="utf-8", errors="ignore") as file:
18     for line in file:
19         try:
20             alert = json.loads(line)
21         except json.JSONDecodeError:
22             continue
23
24         rule = alert.get("rule", {})
25         level = rule.get("level", 0)
26
27         if level >= minimum_level:
28             matches.append(alert)
29
30 for alert in matches[-limit:]:
31     rule = alert.get("rule", {})
32     agent = alert.get("agent", {})
33
34     print(f"Time: {alert.get('timestamp')}")
35     print(f"Agent: {agent.get('name', 'N/A')}")
36     print(f"Agent ID: {agent.get('id', 'N/A')}")
37     print(f"Rule ID: {rule.get('id')}")
38     print(f"Level: {rule.get('level')}")
39     print(f>Description: {rule.get('description')}")
40     print("-" * 70)
41
42 if not matches:
43     print("No high-severity alerts found.")

```

```

(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ python high-severity-alerts.py

High Severity Alerts
=====
No high-severity alerts found.
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$

```

Next, I configured **High-Severity Alert Monitoring**: Created a Python script to identify and report high-severity alerts from Wazuh alert logs.

```
.env test.py agent-inventory.py offline-agents.py high-severity-alerts.py soc-report.py X
soc-report.py
1 import json
2 import requests
3 from pathlib import Path
4 from datetime import datetime
5
6 wazuh_url = "https://localhost:55000"
7 wazuh_user = "wazuh"
8 wazuh_pass = "wazuh"
9
10 alerts_file = Path("/var/ossec/logs/alerts/alerts.json")
11 output_file = Path("daily_soc_report.txt")
12
13 requests.packages.urllib3.disable_warnings()
14
15
16 def get_token():
17     response = requests.post(
18         f"{wazuh_url}/security/user/authenticate",
19         auth=(wazuh_user, wazuh_pass),
20         verify=False
21     )
22     response.raise_for_status()
23     return response.json()["data"]["token"]
24
25
26 def get_agents():
27     token = get_token()
28     headers = {"Authorization": f"Bearer {token}"}
29
30     response = requests.get(
31         f"{wazuh_url}/agents",
32         headers=headers,
33         verify=False
34     )
35     response.raise_for_status()
36     return response.json()["data"]["affected_items"]
37
38
39 def get_high_alerts(minimum_level=10, limit=20):
40     alerts = []
41
42     if not alerts_file.exists():
43         return alerts
44
```

```
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ python soc-report.py
Report created: /home/wazuh/Desktop/wazuh-python-automation/daily_soc_report.txt
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ cat daily_soc_report.txt
Wazuh Daily SOC Report
=====
Generated: 2026-06-13 09:19:37.254516

Agent Summary
-----
Total Agents: 4
Active Agents: 3
Inactive Agents: 1

Inactive Agents
-----
ID: 004 | Name: WIN11PC | IP: [REDACTED] | Status: pending

High Severity Alerts
-----
No high-severity alerts found.(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$
```

The following images demonstrate the testing of a **Daily SOC Report Generation**: a Python script that authenticates to the Wazuh API, collects agent and alert data, and generates a consolidated daily SOC report for security monitoring.

```
.env test.py agent-inventory.py offline-agents.py high-severity-alerts.py soc-report.py wazuh_client.py X
wazuh_client.py
1 import json
2 from pathlib import Path
3
4 import requests
5
6 WAZUH_URL = "https://localhost:55000"
7 WAZUH_USER = "wazuh"
8 WAZUH_PASS = "wazuh"
9
10 ALERTS_FILE = Path("/var/ossec/logs/alerts/alerts.json")
11
12 requests.packages.urllib3.disable_warnings()
13
14
15 def get_token():
16     response = requests.post(
17         f"{WAZUH_URL}/security/user/authenticate",
18         auth=(WAZUH_USER, WAZUH_PASS),
19         verify=False
20     )
21     response.raise_for_status()
22     return response.json()["data"]["token"]
23
24
25 def get_headers():
26     token = get_token()
27     return {"Authorization": f"Bearer {token}"}
28
29
30 def get_agents():
31     response = requests.get(
32         f"{WAZUH_URL}/agents",
33         headers=get_headers(),
34         verify=False
35     )
36     response.raise_for_status()
37     return response.json()["data"]["affected_items"]
```

I then proceeded further with the wazuh client module development, a centralized Python module to handle Wazuh API authentication, agent retrieval, alert processing, and shared functionality used throughout the automation project.

```
.env test.py agent-inventory.py offline-agents.py high-severity-alerts.py soc-report.py wazuh_client.py test2.py test_client.py X
test_client.py
1 from wazuh_client import get_agents, get_offline_agents, get_high_severity_alerts
2
3 print("Agents:")
4 for agent in get_agents():
5     print(agent.get("id"), agent.get("name"), agent.get("status"))
6
7 print("\nOffline Agents:")
8 offline_agents = get_offline_agents()
9
10 if offline_agents:
11     for agent in offline_agents:
12         print(agent.get("id"), agent.get("name"), agent.get("status"))
13 else:
14     print("All agents are active.")
15
16 print("\nHigh Severity Alerts:")
17 alerts = get_high_severity_alerts()
18
19 if alerts:
20     for alert in alerts:
21         print(alert.get("timestamp"), alert.get("rule", {}).get("description"))
22 else:
23     print("No high-severity alerts found.")
```

A test script was then made to validate agent inventory retrieval, offline agent detection, and high-severity alert functions from the centralized Wazuh client module.

```
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ python test_client.py
Agents:
000 wazuh active
002 SRV2016 active
003 SRV2019 active
004 WIN11PC pending

Offline Agents:
004 WIN11PC pending

High Severity Alerts:
No high-severity alerts found.
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$
```

I then ran the test_client script to ensure full functionality of it.

Part 2: MCP

```
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ pip install mcp
Collecting mcp
  Downloading mcp-1.27.2-py3-none-any.whl (220 kB)
    220.5/220.5 KB 5.0 MB/s eta 0:00:00
Collecting pyjwt[crypto]>=2.10.1
  Downloading pyjwt-2.13.0-py3-none-any.whl (31 kB)
Collecting typing-extensions>=4.9.0
  Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
    44.6/44.6 KB 20.8 MB/s eta 0:00:00
Collecting python-multipart>=0.0.9
  Downloading python_multipart-0.0.32-py3-none-any.whl (30 kB)
Collecting pydantic<3.0.0,>=2.11.0
  Downloading pydantic-2.13.4-py3-none-any.whl (472 kB)
    472.3/472.3 KB 23.2 MB/s eta 0:00:00
Collecting starlette>=0.27
  Downloading starlette-1.3.1-py3-none-any.whl (73 kB)
    73.6/73.6 KB 30.2 MB/s eta 0:00:00
Collecting anyio>=4.5
  Downloading anyio-4.13.0-py3-none-any.whl (114 kB)
    114.4/114.4 KB 53.6 MB/s eta 0:00:00
Collecting pydantic-settings>=2.5.2
  Downloading pydantic_settings-2.14.1-py3-none-any.whl (60 kB)
    61.0/61.0 KB 26.7 MB/s eta 0:00:00
Collecting httpx<1.0.0,>=0.27.1
```

Next, I ran the command **pip install mcp** to install the MCP server on the ubuntu vm.

```
mcp_server.py X
mcp_server.py
1  from mcp.server.fastmcp import FastMCP
2
3  from wazuh_client import (
4      get_agents,
5      get_offline_agents,
6      get_high_severity_alerts,
7  )
8
9  mcp = FastMCP("wazuh-soc-mcp")
10
11
12 @mcp.tool()
13 def list_agents() -> List:
14     """List all Wazuh agents with ID, name, IP, and status."""
15     agents = get_agents()
16
17     return [
18         {
19             "id": agent.get("id"),
20             "name": agent.get("name"),
21             "ip": agent.get("ip", "N/A"),
22             "status": agent.get("status"),
23         }
24         for agent in agents
25     ]
26
27
28 @mcp.tool()
29 def list_offline_agents() -> List:
30     """List Wazuh agents that are not active."""
31     agents = get_offline_agents()
32
33     return [
34         {
35             "id": agent.get("id"),
36             "name": agent.get("name"),
37             "ip": agent.get("ip", "N/A"),
38             "status": agent.get("status"),
39         }
40         for agent in agents
41     ]
42
43
44 (venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ python3 mcp_server.py
```

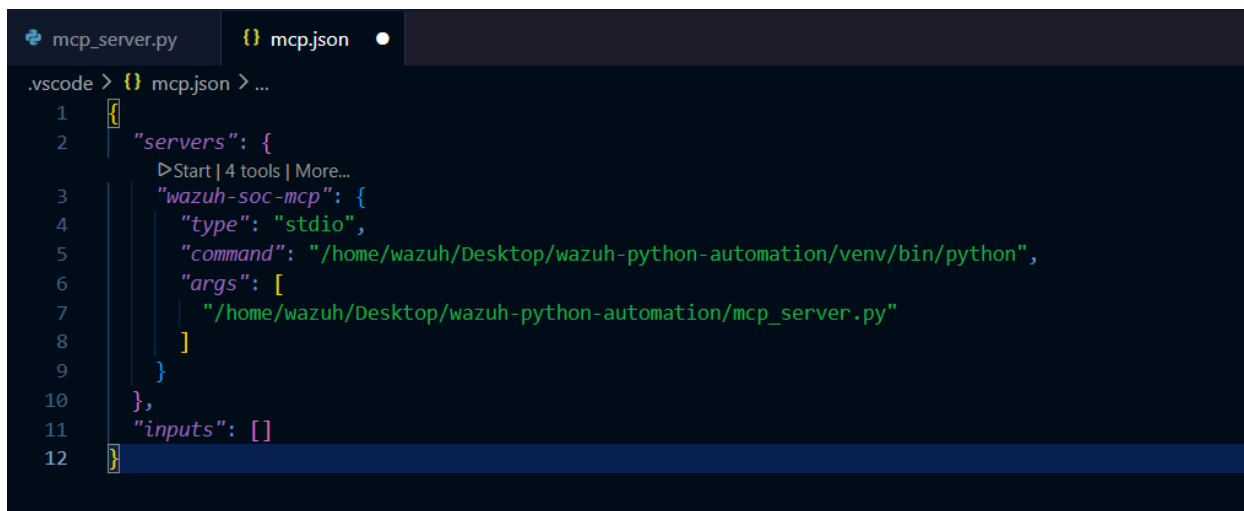
When initially launched, the MCP server produced no console output, which is expected behavior because it waits for incoming tool requests.


```

43 mcp_server.py
44 @mcp.tool()
45 def list_high_severity_alerts(minimum_level: int = 10, limit: int = 10) -> list:
46     """List recent high-severity Wazuh alerts."""
47     alerts = get_high_severity_alerts(
48         minimum_level=minimum_level,
49         limit=limit
50     )
51
52     results = []
53
54     for alert in alerts:
55         rule = alert.get("rule", {})
56         agent = alert.get("agent", {})
57
58         results.append(
59             {
60                 "timestamp": alert.get("timestamp"),
61                 "agent": agent.get("name", "N/A"),
62                 "agent_id": agent.get("id", "N/A"),
63                 "rule_id": rule.get("id"),
64                 "level": rule.get("level"),
65                 "description": rule.get("description"),
66             }
67         )
68
69     return results
70
71 def get_agent_health_summary():
72     agents = get_agents()
73
74     total_agents = len(agents)
75
76     active_agents = [
77         a for a in agents
78         if a.get("status", "").lower() == "active"
79     ]
80
81     offline_agents = [
82         a for a in agents
83         if a.get("status", "").lower() != "active"
84     ]
85
86     return {
87         "total_agents": total_agents,
88         "active_agents": len(active_agents),
89         "offline_agents": len(offline_agents),
90         "offline_agent_names": [
91             agent.get("name")
92             for agent in offline_agents
93         ]
94     }

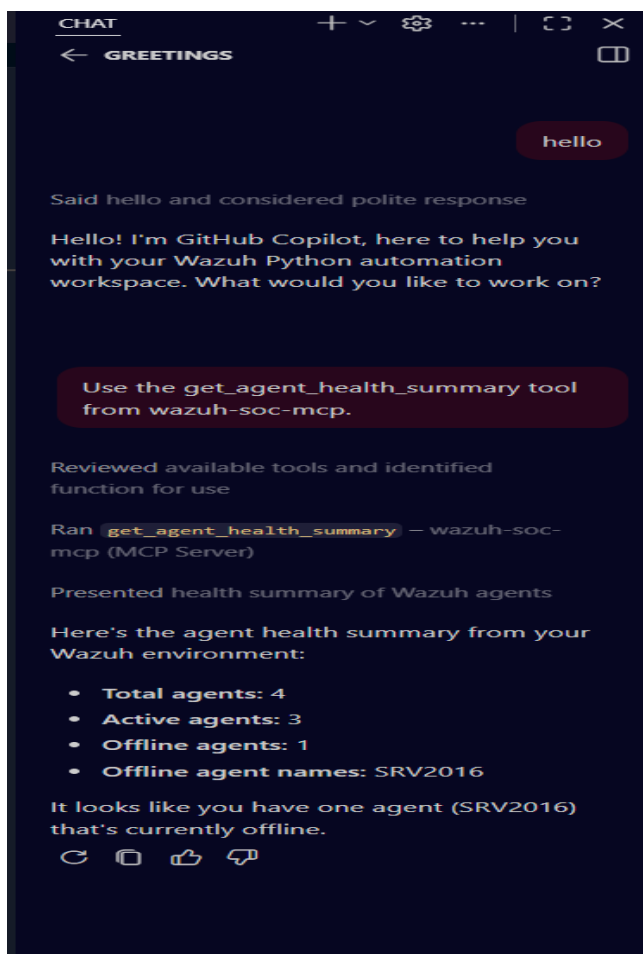
```

This is part of the FastMCP server script with a few more functions that once I got the MCP server running, I wanted it to test.



```
.vscode > {} mcp.json > ...
1  {}
2  "servers": {
3      ▶Start | 4 tools | More...
4      "wazuh-soc-mcp": {
5          "type": "stdio",
6          "command": "/home/wazuh/Desktop/wazuh-python-automation/venv/bin/python",
7          "args": [
8              "/home/wazuh/Desktop/wazuh-python-automation/mcp_server.py"
9          ]
10     },
11     "inputs": []
12 }
```

Next, I created an mcp.json file I can go to for when I want to start running the MCP server defining the server runtime, Python environment, and execution path in the mcp.json configuration file.



Next in VSCode, I typed `ctrl + shift + p`, and typed Chat: Open Chat, which opened this sidebar here of things I can ask or tell it to do. This shows an end-to-end workflow that combines Wazuh monitoring, Python automation, MCP integration, and AI-assisted security operations. The resulting solution allows security information to be queried, summarized, and reported through natural language interactions while maintaining direct integration with the underlying Wazuh platform.



```
114
115 def generate_daily_soc_report():
116     agents = get_agents()
117     offline_agents = get_offline_agents()
118     high_alerts = get_high_severity_alerts(minimum_level=10, limit=20)
119     recent_counts = get_recent_alert_count(hours=24)
120
121     active_agents = []
122     for agent in agents:
123         if agent.get("status", "").lower() == "active":
124             active_agents.append(agent)
125
126     lines = []
127
128     lines.append("Wazuh Daily SOC Report")
129     lines.append("-" * 60)
130     lines.append(f"Generated: {datetime.now()}")
131     lines.append("")
132
133     lines.append("Agent Summary")
134     lines.append(f"Total Agents: {len(agents)}")
135     lines.append(f"Active Agents: {len(active_agents)}")
136     lines.append(f"Inactive Agents: {len(offline_agents)}")
137     lines.append("")
138
139     lines.append("Recent Alert Summary")
140     lines.append("-" * 60)
141     lines.append(f"Alert Window: Last {recent_counts['hours']} hours")
142     lines.append(f"Total Alerts: {recent_counts['total_alerts']}")
143     lines.append(f"High Severity Alerts: {recent_counts['high_severity_alerts']}")
144     lines.append("")
145
146     lines.append("Inactive Agents")
147     lines.append("-" * 60)
148
149     if offline_agents:
150         for agent in offline_agents:
151             lines.append(
152                 f"""ID: {agent.get('id')} | "
153                 f"""Name: {agent.get('name')} | "
154                 f"""IP: {agent.get('ip', 'N/A')} | "
155                 f"""Status: {agent.get('status')}"""
156             )
157     else:
158         lines.append("No inactive agents found.")
159
160     lines.append("")
161     lines.append("High Severity Alerts")
162     lines.append("-" * 60)
163
164     # High severity alerts
```

Use the generate_daily_soc_report_tool from wazuh-soc-mcp.

Fixed recursion issue in mcp_server.py

Ran generate_daily_soc_report - wazuh-soc-mcp (MCP Server)

Fixed recursion error and confirmed tool functionality

Perfect! The tool is now working correctly. Here's the daily SOC report summary:

Agent Summary:

- Total Agents: 4
- Active Agents: 3
- Inactive Agents: 1 (SRV2016)

Recent Alerts (Last 24 hours):

- Total Alerts: 0
- High Severity Alerts: 0 (note: there are 20 high-severity CVE alerts listed separately)

Inactive Agents:

- SRV2016 (ID: 002) - IP: 192.168.16.171 - Status: disconnected

High Severity Alerts (20 detected):

- All from WIN10PC with Rule ID 23505 (Level 10)
- Multiple CVE vulnerabilities affecting Windows 11 Pro including CVE-2025-21277, CVE-2025-58735, CVE-2024-49095, and 17 others

1 file changed +4 -4 Keep Undo

Describe what to build

Claude Hailu 4.5

This displays lastly the development of a Wazuh-powered security monitoring and automation platform that combines SIEM functionality, Python-based operational tooling, and MCP-enabled AI integration. The project demonstrates practical application of cybersecurity automation, endpoint monitoring, alert analysis, SOC reporting, and AI-assisted security operations workflows. This project demonstrates how traditional SIEM monitoring can be enhanced through Python automation and AI integration, reducing manual effort while improving accessibility to security information.